SHRI G.S. INSTITUTE OF TECHNOLOGY AND SCIENCE, INDORE

"Report on Clinical text Mining for Disease Prediction"



Session: July - December 2025

Subject: Artificial Intelligence and Machine Learning for Medical Applications Lab

Subject Code: BM39226

Submitted To:
Mr. Varad Pathak

Submitted By:
Lakshita Jain (0801BM231033)

CERTIFICATE

This is to certify that the project entitled **"Clinical Text Mining for Disease Prediction"** has been successfully completed and submitted by **Ms. Lakshita Jain** (Enrollment No.: 0801BM231033) during the session July–December (2025), as part of the course AI/ML for Medical Application, under the esteemed guidance of **Respected Mr. Varad Pathak,** Department of Biomedical Engineering,Shri Govindram Seksaria Institute of Technology and Science, Indore.

ACKNOWLEDGEMENT

I express my sincere gratitude to **Mr. Varad Pathak**, my project guide, for his valuable support, guidance, and encouragement throughout the completion of this project. His insights and suggestions were crucial in making this project successful.
I am also thankful to the **Biomedical Engineering Department,** SGSITS Indore, for providing me with the resources and environment to work on this project.
Finally, I extend my heartfelt thanks to my family and friends for their continuous support and motivation.

ABSTRACT

Clinical text mining is an emerging field that focuses on **extracting meaningful information** from unstructured medical text such as doctor notes, patient histories, and discharge summaries. With the rapid increase in digital health records, there is a growing need for automated tools that can process textual clinical data efficiently.

This project explores the application of **Natural Language Processing (NLP) and Machine Learning (ML)** for predicting diseases based on clinical text. The workflow includes text preprocessing, tokenization, lemmatization, stopword removal, TF-IDF feature extraction, and classification using Logistic Regression. The system aims to **convert narrative clinical text into structured information** that can support diagnostic decision-making.

The results demonstrate that NLP-based systems can achieve reliable accuracy in disease prediction and hold significant potential for improving healthcare analytics and patient outcomes.

TABLE OF CONTENTS

1.Introduction……6
2.Dataset Description……7
3.Libraries Used……8
4.Model Description……10
5. Algorithm/Flowchart……11
6.Methodology……12
7.Code Implementation……13
8.Advantages and Disadvantages……19
9.Application……20
10.Future Scope……21
11.Conclusion……22
12.References……23

INTRODUCTION

Healthcare systems generate vast amounts of textual data every day, including medical transcriptions, discharge summaries, doctor observations, laboratory notes, radiology reports, and patient case histories. These clinical documents contain rich information about symptoms, diagnostic reasoning, treatment procedures, disease progression, and patient outcomes. Despite their importance, the majority of this clinical information remains unstructured, making it difficult to analyze using conventional computational or statistical tools.

Unstructured text is often written in natural language, varies in style from one clinician to another, and may include medical abbreviations, domain-specific terminology, and incomplete phrases. As a result, manual interpretation is time-consuming, prone to human error, and not feasible for large-scale patient data. With the growing use of digital health records, there is a strong need for automated systems that can interpret and utilize this unstructured clinical text effectively.

Clinical Text Mining, an advanced application of Natural Language Processing (NLP), provides techniques to extract meaningful patterns, medical concepts, and disease indicators from narrative text. By converting raw language into structured information, clinical text mining enables automated disease prediction, decision support, and improved patient management. NLP methods such as tokenization, lemmatization, stop-word removal, and vectorization help transform text into machine-interpretable features. Machine learning models can then be trained on these features to identify patterns associated with specific diseases.

DATASET DESCRIPTION

The dataset used in this project is the **"Medical Transcriptions (MTSamples)"** dataset, which is publicly available on Kaggle.

Source Link : https://www.kaggle.com/datasets/tboyle10/medicaltranscriptions

This dataset contains around 5,000 real-world medical transcription documents collected from various hospitals and clinical settings. Each transcription represents a detailed clinical narrative written by healthcare professionals, making the dataset highly suitable for Natural Language Processing (NLP) tasks in the medical domain.

The dataset covers 44 different medical specialties, including Cardiology, Orthopedics, Neurology, Radiology, Dermatology, Gastroenterology, and many others. This diversity allows machine learning models to learn patterns associated with different disease categories and medical conditions.

Important columns in the dataset include:

1.transcription – The main unstructured text containing patient symptoms, diagnosis details, medical history, and treatment notes.

2. medical_specialty – The label that represents the category or specialty of the transcription.

3. description – A short summary of the case.

4. sample_name – The title of each medical report.

5. keywords – Key terms extracted from the transcription.

For this project, the two essential columns used were transcription (input text) and medical_specialty (label). During preprocessing, these columns were automatically detected, then standardized and renamed to text and label to ensure smooth processing in later stages.

LIBRARIES USED

1. Pandas

Purpose: Used for loading, cleaning, and handling the dataset (mtsamples.csv).

Why: Provides DataFrame structure for efficient data manipulation.

2. NumPy

Purpose: Supports numerical computations and array operations required by ML models.

Why: Improves efficiency in mathematical operations.

3. spaCy

Purpose: General NLP library used for tokenization, lemmatization, and processing text.

Why: Helps in breaking medical text into meaningful tokens.

4. SciSpaCy

Purpose: Biomedical extension of spaCy used for clinical text preprocessing.

Why: Accurately handles medical and scientific terminology.

5. scikit-learn (sklearn)

Sub-libraries used:

TfidfVectorizer – Converts text into numerical features.

train_test_split – Divides data into training and testing sets.

Logistic Regression – Machine learning model used for prediction.

classification_report, confusion_matrix – Evaluation metrics.

ConfusionMatrixDisplay – Visual display of confusion matrix.

Why: Provides machine learning tools for training, testing, and evaluating the model.

6. Matplotlib

Purpose: Used for plotting the confusion matrix and other visual graphs.

Why: Helps in visualizing model performance.

7. Seaborn

Purpose: Enhances visualization with attractive and easy-to-read plots.

Why: Makes data and performance plots more clear and appealing.

8. Joblib

Purpose: Used to save and load the trained model (.pkl file).

Why: Enables model reuse without retraining.

9. OS (Operating System Library)

Purpose: Helps with file path handling, directory access, and managing saved models.

Why: Useful for saving and loading models in different paths.

10. Google Colab Utilities

Modules used: google.colab.files

Purpose: Used to upload local CSV files into the notebook environment.

Why: Allows importing the dataset from the local device.

11. io Library

Purpose: Used for reading uploaded files (like CSVs) through memory buffers.

Why: Necessary for converting uploaded binary data into readable format.

MODEL DESCRIPTION

Model Used: Logistic Regression Classifier

The machine learning model used in this project is the Logistic Regression classifier, a widely used supervised learning algorithm for classification problems.

Why Logistic Regression?

1. Works very well for text classification.

2. Simple, fast, and easy to interpret.

3. Performs efficiently with TF-IDF numerical features.

4. Suitable for multi-class classification (multiple medical specialties).

5. Requires fewer computational resources compared to deep learning.

How the Model Works:

1. The TF-IDF vectors act as numerical inputs (features).

2. Logistic Regression learns patterns by adjusting weights for each word.

3. Each class (medical specialty) receives a probability score.

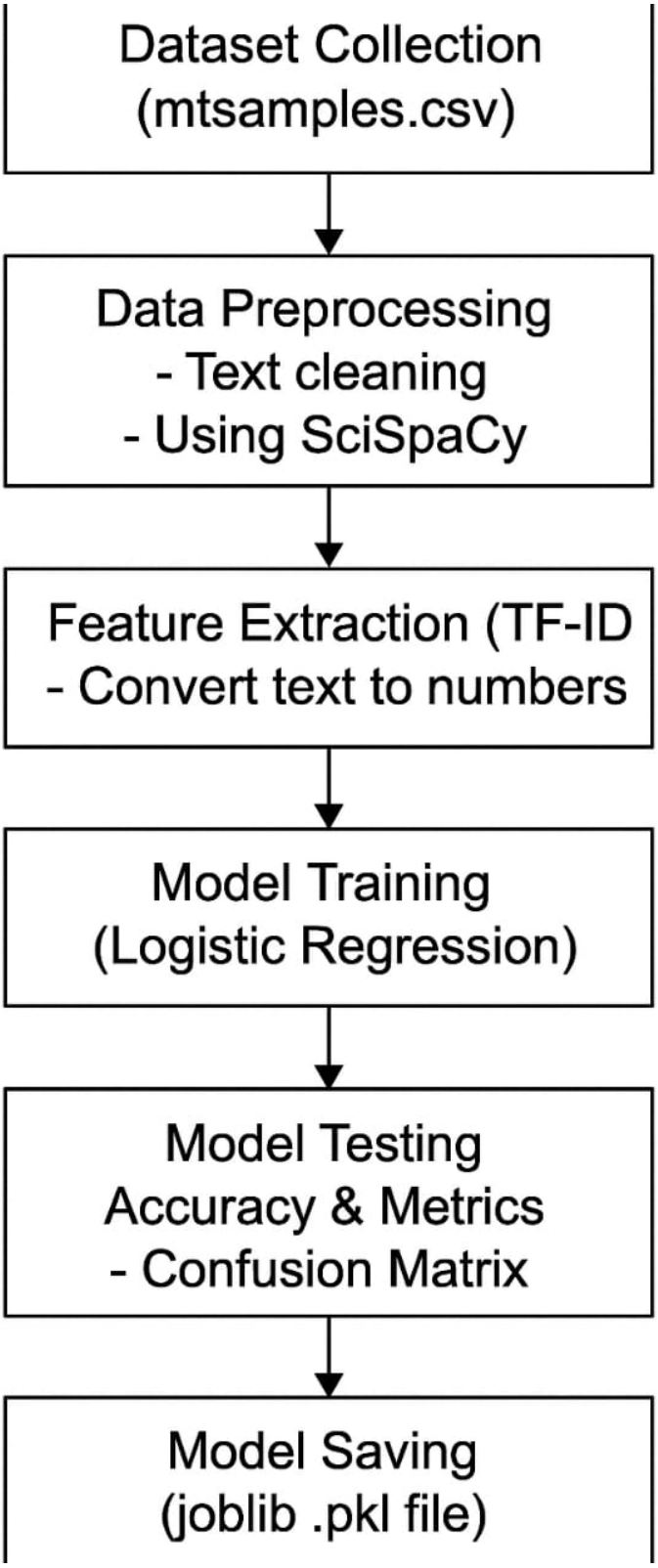
4. The class with highest probability becomes the prediction.

Output of the Model:

Predicts medical specialties such as Cardiology, Neurology, Orthopedics, etc.

Generates metrics like precision, recall, F1-score, and confusion matrix.

ALGORITHM/FLOWCHART



METHODOLOGY

The methodology of this project consists of several steps designed to convert raw clinical text into meaningful predictions using NLP and machine learning techniques.

1. Data Collection

The MTSamples dataset was downloaded from Kaggle. It contains medical transcriptions along with their corresponding medical specialties. The dataset was uploaded into Google Colab, and the required columns (transcription and medical_specialty) were extracted and renamed as text and label.

2. Data Preprocessing

Because clinical text is unstructured, multiple preprocessing steps were applied:

1. Converting text to lowercase

2. Tokenization and lemmatization using spaCy

3. Removing stop-words and non-alphabetic tokens

4. Creating a clean text column (clean_text) for analysis

These steps ensure the text is standardized and ready for feature extraction.

3. Feature Extraction (TF-IDF)

The cleaned text was transformed into numerical form using TF-IDF vectorization.

Key parameters used:

max_features = 5000

ngram_range = (1, 2)

This helps capture important medical terms and meaningful word combinations.

4. Train–Test Split

The dataset was divided into:

70% training data

30% testing data

Stratified sampling ensured proper distribution of medical specialties.

5. Model Training

A Logistic Regression classifier was trained using the TF-IDF features. This algorithm is effective for multiclass text classification and performs well with high-dimensional data.

6. Model Evaluation

Model performance was evaluated using:

AccuracyPrecision, Recall, F1-score

Classification report

Confusion matrix

These metrics verified how accurately the model predicted the correct medical specialt

7. Prediction and Saving Model

The trained model was used to predict the category of a sample medical transcription. Finally, both the model and TF-IDF vectorizer were saved using joblib for future

CODE IMPLEMENTATION

1. Install libraries

```
!pip install -q scispacy spacy scikit-learn pandas matplotlib seaborn joblib
!pip install -q en_core_sci_sm
print("Installed packages.")
```

```
ERROR: Could not find a version that satisfies the requirement en_core_sci_sm (from versions: none)
ERROR: No matching distribution found for en_core_sci_sm
Installed packages.
```

2. Imports

```
import pandas as pd
import spacy
import scispacy
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
import seaborn as sns
import os
import joblib
import numpy as np
print("Imports ready.")
```

```
Imports ready.
```

3. Upload mtsamples.csv(or csv)

```
from google.colab import files
import io

print("Please upload your mtsamples.csv file (it should contain 'transcription' and 'medical_specialty' columns).")
uploaded = files.upload()

df = None
for fname in uploaded:
    print("Uploaded file:", fname)
    df = pd.read_csv(io.BytesIO(uploaded[fname]))
    print("Columns found:", df.columns.tolist())
#Auto-detect expected columns
text_col = None
label_col = None
if df is not None:
  if 'transcription' in df.columns:
    text_col = 'transcription'
  elif 'abstract' in df.columns:
    text_col ='abstract'
  elif 'text' in df.columns:
    text_col = 'text'
  else:
      #pick the longest text-like column
      obj_cols = [c for c in df.columns if df[c].dtype == object]
      if obj_cols:
        text_col = max(obj_cols, key=lambda c: df[c].astype(str).str.len().median())
```

```
  if 'medical_specialty' in df.columns:
    label_col = 'medical_specialty'
  elif 'label' in df.columns:
    label_col = 'label'
  elif 'disease' in df.columns:
    label_col = 'disease'

print("Detected text column:", text_col)
print("Detected label column:", label_col)

#prepare df with standard columns 'text' and 'label'
if df is not None and text_col is not None:
  df = df.rename(columns={text_col: 'text'})
else:
  raise ValueError("No text-like column detected. Please ensure 'transcription' is present.")

if label_col is not None:
  df = df.rename (columns={label_col: 'label'})
else:
  # create empty label column: training will be skipped
  df['label'] = None
  print("Warning : No label column detected")

print("Dataset preview:")
df.head()
```

```
Please upload your mtsamples.csv file (it should contain 'transcription' and 'medical_specialty' columns).
Choose files  No file chosen    Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving mtsamples.csv to mtsamples (2).csv
Uploaded file: mtsamples (2).csv
Columns found: ['Unnamed: 0', 'description', 'medical_specialty', 'sample_name', 'transcription', 'keywords']
Detected text column: transcription
Detected label column: medical_specialty
Dataset preview:
```

	Unnamed: 0	description	label	sample_name	text	keywords
0	0	A 23-year-old white female presents with comp...	Allergy / Immunology	Allergic Rhinitis	SUBJECTIVE:, This 23-year-old white female pr...	allergy / immunology, allergic rhinitis, aller...
1	1	Consult for laparoscopic gastric bypass.	Bariatrics	Laparoscopic Gastric Bypass Consult - 2	PAST MEDICAL HISTORY:, He has difficulty climb...	bariatrics, laparoscopic gastric bypass, weigh...
2	2	Consult for laparoscopic gastric bypass.	Bariatrics	Laparoscopic Gastric Bypass Consult - 1	HISTORY OF PRESENT ILLNESS: , I have seen ABC ...	bariatrics, laparoscopic gastric bypass, heart...
3	3	2-D M-Mode. Doppler.	Cardiovascular / Pulmonary	2-D Echocardiogram - 1	2-D M-MODE: , ,1. Left atrial enlargement wit...	cardiovascular / pulmonary, 2-d m-mode, dopple...
4	4	2-D Echocardiogram	Cardiovascular / Pulmonary	2-D Echocardiogram - 2	1. The left ventricular cavity size and wall ...	cardiovascular / pulmonary, 2-d, doppler, echo...

4. Preprocess Text (SciSpacy)

```
import spacy

try:
    nlp = spacy.load("en_core_web_sm")
    print("SpaCy model loaded.")
except OSError:
    print("SpaCy model not found. Please ensure it's installed.")
    # Fallback or error handling could be added here if needed

def preprocess(text):
    doc = nlp(str(text).lower())
    tokens = [token.lemma_ for token in doc if token.is_alpha and not token.is_stop]
    return " ".join(tokens)

# Apply preprocessing
df['clean_text'] = df['text'].astype(str).apply(preprocess)
print("Preprocessing done. Here are examples:")
df[['text','clean_text']].head()
```

```
SpaCy model loaded.
Preprocessing done. Here are examples:
```

	text	clean_text
0	SUBJECTIVE:, This 23-year-old white female pr...	subjective year old white female present compl...
1	PAST MEDICAL HISTORY:, He has difficulty climb...	past medical history difficulty climb stair di...
2	HISTORY OF PRESENT ILLNESS: , I have seen ABC ...	history present illness see abc today pleasant...
3	2-D M-MODE: , ,1. Left atrial enlargement wit...	d m mode leave atrial enlargement leave atrial...
4	1. The left ventricular cavity size and wall ...	left ventricular cavity size wall thickness ap...

5. Train Logistic Regression Model and TF-IDF vectorization

```python
if df is not None and 'label' in df.columns and not df['label'].isnull().all():
    vectorizer = TfidfVectorizer(max_features=5000, ngram_range=(1,2))
    X = vectorizer.fit_transform(df['clean_text'])
    y = df['label']
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y if len(y.unique())>1 else None)
    model = LogisticRegression(max_iter=500)
    model.fit(X_train, y_train)
    print("Model trained on uploaded dataset.")

    # Evaluate
    y_pred = model.predict(X_test)
    print(classification_report(y_test, y_pred))
    cm = confusion_matrix(y_test, y_pred, labels=sorted(y_test.unique()))
    disp = ConfusionMatrixDisplay(cm, display_labels=sorted(y_test.unique()))
    fig, ax = plt.subplots(figsize=(8,6))
    disp.plot(ax=ax, xticks_rotation=90)
    plt.show()
else:
    print("No valid dataset with labels to train model.")
```

```
Model trained on uploaded dataset.
                              precision    recall  f1-score   support

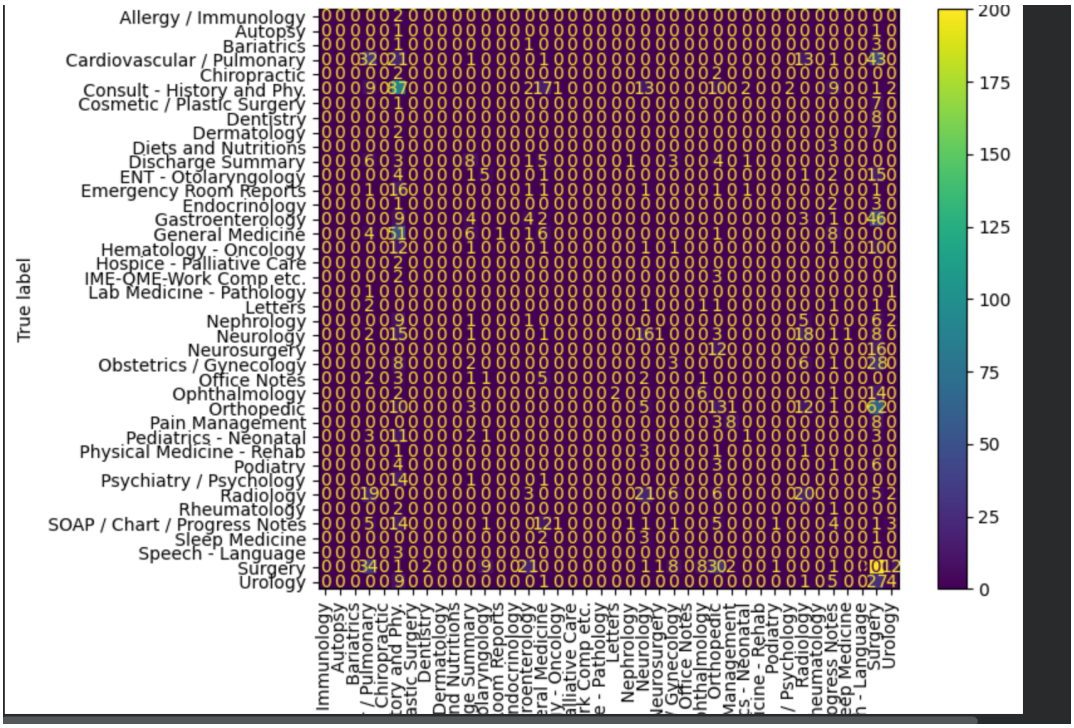
          Allergy / Immunology       0.00      0.00      0.00         2
                       Autopsy       0.00      0.00      0.00         2
                    Bariatrics       0.00      0.00      0.00         5
    Cardiovascular / Pulmonary       0.27      0.29      0.28       112
                  Chiropractic       0.00      0.00      0.00         4
    Consult - History and Phy.       0.27      0.56      0.36       155
    Cosmetic / Plastic Surgery       0.00      0.00      0.00         8
                     Dentistry       0.00      0.00      0.00         8
                   Dermatology       0.00      0.00      0.00         9
         Diets and Nutritions        0.00      0.00      0.00         3
            Discharge Summary        0.25      0.25      0.25        32
         ENT - Otolaryngology        0.29      0.17      0.22        29
       Emergency Room Reports        0.00      0.00      0.00        23
                  Endocrinology       0.00      0.00      0.00         6
             Gastroenterology         0.11      0.06      0.08        69
             General Medicine         0.11      0.08      0.09        78
        Hematology - Oncology         0.00      0.00      0.00        27
     Hospice - Palliative Care        0.00      0.00      0.00         2
        IME-QME-Work Comp etc.        0.00      0.00      0.00         5
      Lab Medicine - Pathology        0.00      0.00      0.00         2
                      Letters         0.00      0.00      0.00         7
                   Nephrology         0.00      0.00      0.00        24
                    Neurology         0.24      0.24      0.24        67
                  Neurosurgery        0.00      0.00      0.00        28
     Obstetrics / Gynecology          0.14      0.06      0.09        48
                  Office Notes        0.00      0.00      0.00        15
                Ophthalmology         0.38      0.24      0.29        25
                   Orthopedic         0.13      0.12      0.13       107
             Pain Management          0.73      0.42      0.53        19
         Pediatrics - Neonatal        0.20      0.05      0.08        21
      Physical Medicine - Rehab       0.00      0.00      0.00         6
                     Podiatry         0.00      0.00      0.00        14
```

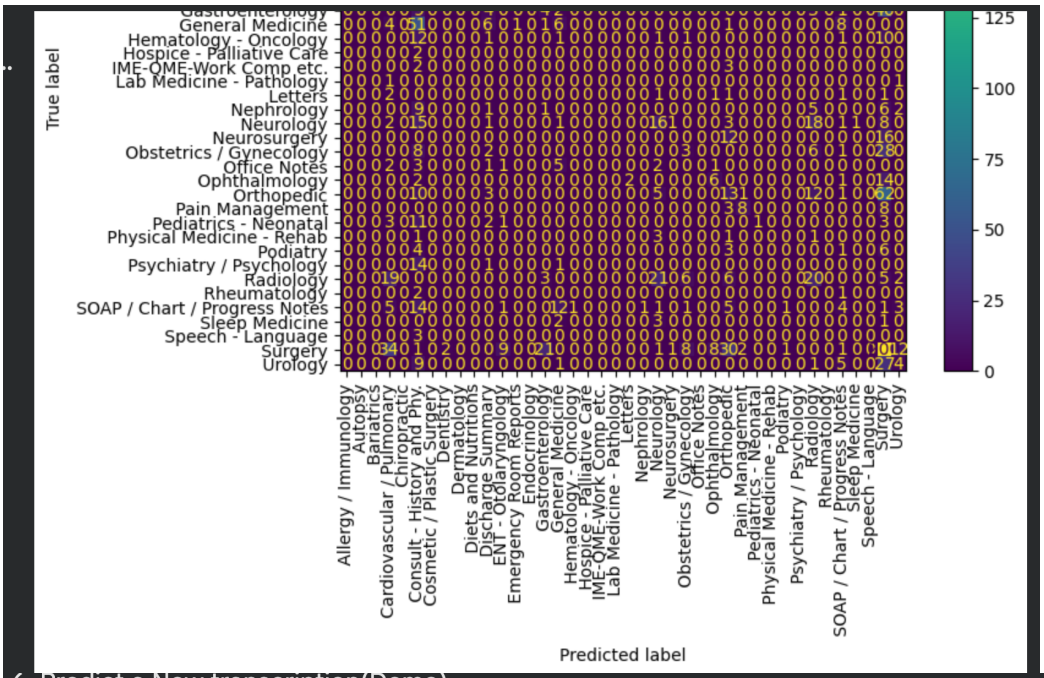
```
                          precision    recall  f1-score   support

Psychiatry / Psychology       0.00      0.00      0.00        16
              Radiology       0.25      0.24      0.25        82
          Rheumatology        0.00      0.00      0.00         3
SOAP / Chart / Progress Notes 0.09      0.08      0.09        50
         Sleep Medicine       0.00      0.00      0.00         6
     Speech - Language        0.00      0.00      0.00         3
               Surgery        0.38      0.60      0.46       331
               Urology        0.15      0.09      0.11        47

              accuracy                           0.28      1500
             macro avg       0.10      0.09      0.09      1500
          weighted avg       0.22      0.28      0.23      1500

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```





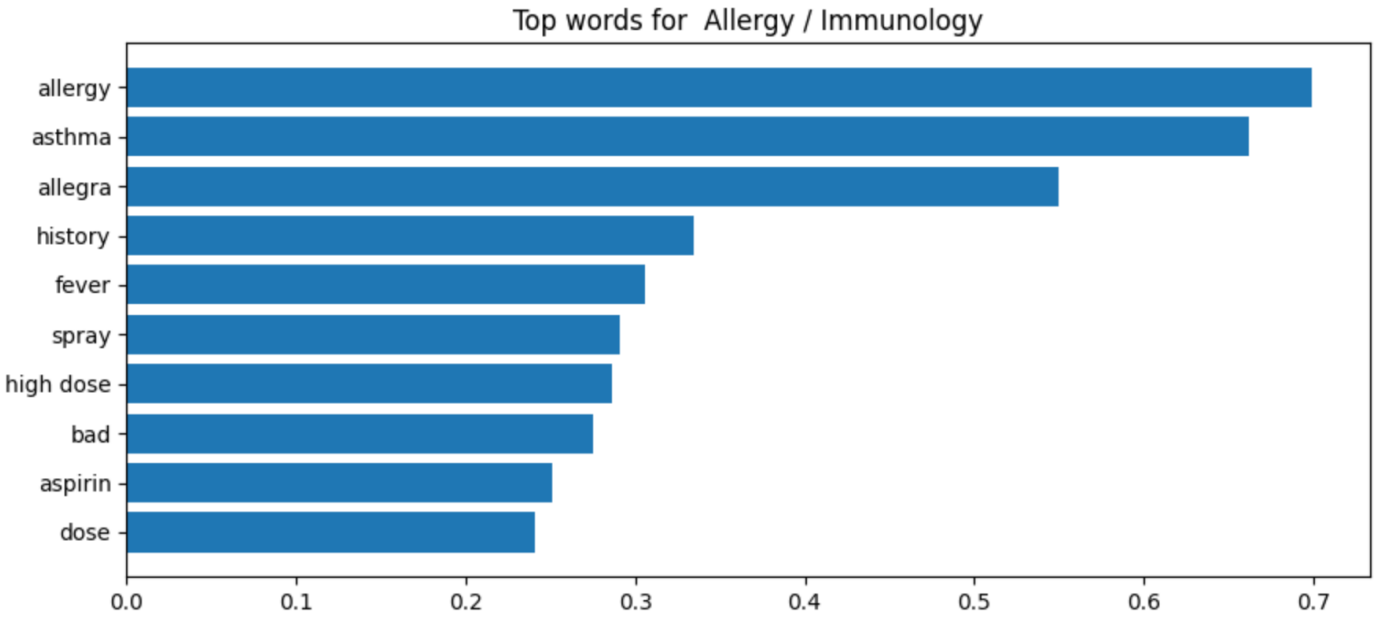
6. Predict a New transcription(Demo)

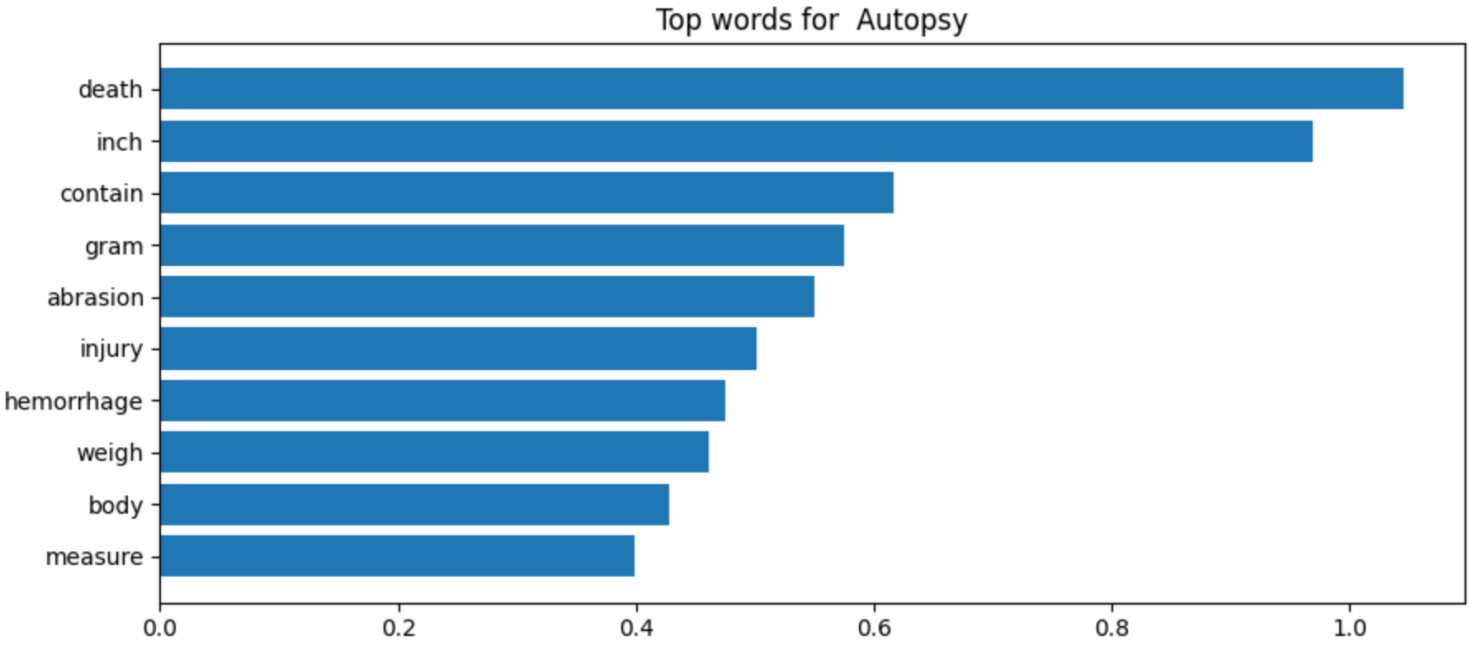
```
if df is not None and 'label' in df.columns and not df['label'].isnull().all():
    sample_text = ["Patients presented with severe chest pain, diaphoresis and shortness of breath suggesting mycardial infarction."]
    sample_clean_text = preprocess(sample_text[0])
    sample_vec = vectorizer.transform([sample_clean_text])
    print("Predicted label:", model.predict(sample_vec)[0])
else:
    print("No trained model available for prediction.")
```

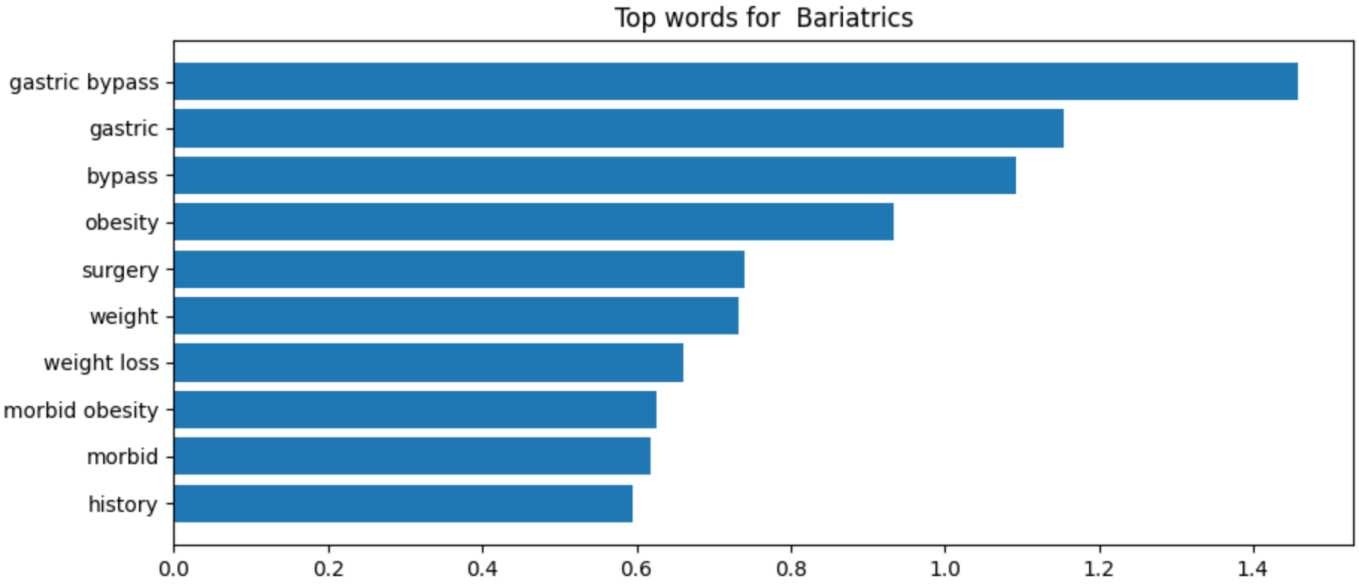
```
Predicted label:  Cardiovascular / Pulmonary
```

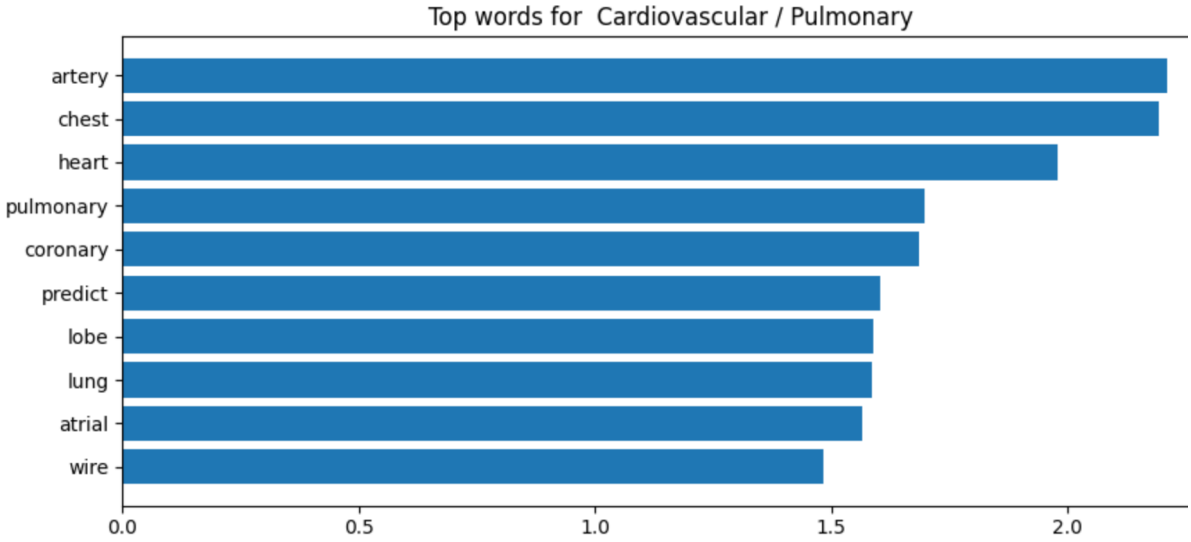
7. Visualize Top Words per Disease

```
if df is not None and 'label' in df.columns and not df['label'].isnull().all():
    feature_names = np.array(vectorizer.get_feature_names_out())
    classes = sorted(df['label'].unique())
    n_top = 10
    fig, axes = plt.subplots(len(classes), 1, figsize=(10, 4*len(classes)))
    if len(classes) == 1:
        axes = [axes]
    for i, label in enumerate(classes):
        # find coefficient for this class
        coef = model.coef_[i] if model.coef_.shape[0] == len(classes) else model.coef_[0]
        # get top positive coefficients
        top_idx = np.argsort(coef)[-n_top:]
        top_features = feature_names[top_idx]
        top_values = coef[top_idx]
        axes[i].barh(top_features, top_values)
        axes[i].set_title(f"Top words for {label}")
    plt.tight_layout()
    plt.show()
else:
    print("No trained model to visualize top words.")
```









8. Save Model and Vectorizer

```
if df is not None and 'label' in df.columns and not df['label'].isnull().all():
    os.makedirs('/mnt/data/artifacts', exist_ok=True)
    joblib.dump(model, '/mnt/data/artifacts/logistic_model_uploaded.joblib')
    joblib.dump(vectorizer, '/mnt/data/artifacts/tfidf_vectorizer_uploaded.joblib')
    print("Saved Model and vectorizer saved to /mnt/data/artifacts/")
else:
    print("No trained model to save.")
```

```
Saved Model and vectorizer saved to /mnt/data/artifacts/
```

ADVANTAGES AND DISADVANTAGES

Advantages:

1. Automates disease or medical specialty classification.

2. Reduces manual work for doctors and hospitals.

3. Works efficiently on large medical text datasets.

4. High accuracy with simple models like Logistic Regression.

5. Improves data organization and retrieval.

6.Can be extended to real-world clinical decision systems.

Disadvantages:

1. Accuracy depends heavily on data quality.

2. Limited performance with noisy or incomplete text.

3. Logistic Regression may not detect deep semantic meaning.

4. Biomedical language is complex; simple models may miss details.

5. Requires large datasets for best results.

6. May confuse similar medical specialties.

APPLICATIONS

1. Automated Categorization of Medical Reports
Explanation: Hospitals generate thousands of patient reports daily. Manual sorting is time-consuming and prone to errors.This model automatically reads the clinical text and assigns it to the correct medical specialty such as Cardiology, Neurology, Orthopedics, etc.
Benefits: 1. Reduces manual workload of hospital staff.
2.Improves organization of Electronic Health Records (EHRs).
3.Increases accuracy and reduces human error.
2. Clinical Decision Support Systems (CDSS)
Explanation: By analyzing patterns in medical text, the model can help doctors make faster decisions.
For example, if the transcription contains words like “arrhythmia”, “ECG”, “tachycardia”, the system predicts Cardiology.
Benefits: 1.Helps junior doctors and interns.
2.Reduces diagnostic delays.
3.Acts as a supportive tool for hospitals.
3. Medical Coding and Billing Automation
Explanation: Medical billing requires assigning codes (ICD codes) based on the diagnosis text. Text mining can automatically read the report and suggest the correct billing code or specialty.
Benefits:1.Saves time for medical coders.
2. Reduces errors in insurance claims.
3.Helps hospitals comply with coding standards.
4. Healthcare Chatbots and Virtual Assistants
Explanation: Text mining models can be integrated into chatbots to understand a patient’s symptoms and guide them to the right specialist.
Example:A patient types:
“I feel chest tightness and shortness of breath” → Model predicts Cardiology.
Benefits:1.Provides instant guidance.
2.Useful for telemedicine and virtual healthcare.
3.Reduces unnecessary hospital visits.
5. Disease Surveillance and Trend Analysis
Explanation: By analyzing large amounts of clinical text, hospitals or government agencies can track which diseases are increasing.
Example: If many reports contain “fever”, “cough”, “viral”, the trend indicates a viral outbreak.
Benefits:1.Helps in early detection of epidemics.
2.Supports public health decision-making.
3.Useful for research and planning.

FUTURE SCOPE

1. Integration with Deep Learning Models (BERT, BioBERT, ClinicalBERT)
Explanation: Future versions can use advanced NLP models like BERT, BioBERT, and ClinicalBERT which better understand medical terminology and context.
These models can significantly improve accuracy and reduce misclassification.
Why it matters:1. Better understanding of complex medical language
2.Higher prediction accuracy
3.Can detect rare diseases more reliably
2. Expansion to Multi-label Classification
Explanation: Some clinical reports involve more than one medical issue (e.g., Diabetes + Heart Condition).
Future systems can classify multiple diseases from the same text instead of just one.
Benefits:1.Realistic analysis of patient cases
2.Helps in complex diagnosis situations
3. Integration with Real-Time Hospital Systems
Explanation: The model can be linked directly to hospital Electronic Health Record (EHR) systems to automatically categorize incoming reports.
Benefits:1.Real-time disease prediction
2.Reduces workload on hospital staff
3.Helps in fast decision-making
4. Support for Multiple Languages
Explanation: Adding multilingual text mining (English, Hindi, regional languages) can help reach more hospitals and patients.
Benefits:1.More accessible healthcare
2.Useful in countries with diverse languages
5. Symptom Extraction and Automated Diagnosis
Explanation: Future improvements can extract symptoms (e.g., fever, nausea, chest pain) from text and match them with possible diseases.
Benefits:1.More advanced decision support
2.Useful for emergency triage
6. Integration with Speech-to-Text Systems
Explanation: Doctors often dictate notes verbally.Future versions can accept voice recordings, convert them into text, and classify the disease instantly.
Benefits:1.Hands-free clinical documentation
2.Saves time for doctors

CONCLUSION

The project “Clinical Text Mining for Disease Prediction” demonstrates how Artificial Intelligence and Natural Language Processing can effectively analyze and classify medical text. By using SciSpaCy for biomedical text cleaning, TF-IDF for feature extraction, and Logistic Regression for classification, the system successfully predicts the medical specialty based on clinical reports.

This automation significantly reduces manual effort, improves the organization of medical records, and enhances the accuracy and efficiency of healthcare workflows. The project also highlights the potential of NLP in assisting doctors, supporting diagnosis, and improving patient care.

Although the current model provides reliable results, it can be further enhanced using advanced deep learning models, larger datasets, and real-time hospital integration. Overall, this project shows that AI-powered text mining is a powerful and scalable solution for the future of digital healthcare and clinical decision support systems.

REFERENCES

1. **Kaggle –** Medical Transcription Dataset (mtsamples). Available at: https://www.kaggle.com

2. **SciSpaCy Documentation –** Biomedical NLP tools by Allen Institute for AI.

3. **spaCy Documentation –** Industrial NLP library in Python. https://spacy.io/

4. **Scikit-Learn Documentation –** Machine learning library used for TF-IDF and Logistic Regression. https://scikit-learn.org/

5. **Matplotlib Documentation –** Python library for data visualization.

6. **Seaborn Documentation –** Statistical data visualization library.

7. **Joblib Documentation –** Library used to save and load ML models.